

Opcode Analiz Aracı: Nesneye Yönelik Programlama İlkeleri ile Sanal Makine Bytecode Analizi

Mert Yasin YILMAZ

Bilgisayar Mühendisliği Bölümü

Kastamonu Üniversitesi, Kastamonu, Türkiye

Özet

Bu çalışmada, C++17 ve Qt 6 çerçevesi kullanılarak geliştirilen nesneye yönelik bir opcode analiz aracı sunulmaktadır. Araç, özel tasarlanmış bir sanal makine bytecode formatını (.opc) analiz etmekte; kapsülleme, kalıtım, polimorfizm ve soyutlama ilkelerini katmanlı bir mimari ile hayata geçirmektedir. Sistem, 25 opcode içeren özel bir talimat seti tanımlar; Factory ve Singleton tasarım örüntülerini kullanan Instruction sınıf hiyerarşisi, Strategy örüntüsü ile uygulanan çoklu analiz stratejileri (frekans, kategori, Shannon entropi, karşılaştırma), operatör aşırı yükleme ve CSV/HTML dışı aktarma destekli Report sınıfı ile eksiksiz bir OOP uygulaması ortaya koymaktadır. Qt 6 tabanlı modern grafik arayüz, açık/koyu tema desteği, interaktif grafikler ve son dosyalar yönetimi sunmaktadır.

Anahtar Kelimeler—Nesneye Yönelik Programlama, C++17, Qt 6, Opcode Analizi, Tasarım Örüntüleri, Bytecode, Shannon Entropi

I. Giriş

Nesneye yönelik programlama (NOP), modern yazılım mühendisliğinin temel paradigmalarından biridir. C++ gibi çok paradigmatlı diller, NOP ilkelerini —kapsülleme, kalıtım, polimorfizm ve soyutlama— performans odaklı sistem programlamasıyla birleştiren güçlü bir platform sunmaktadır.

Bu çalışmada, Nesneye Yönelik Programlama dersinin final projesi kapsamında geliştirilen "Opcode Analiz Aracı" sunulmaktadır. Araç; C++17, Qt 6 GUI çerçevesi ve CMake derleme sistemi kullanılarak inşa edilmiştir. Özel tasarlanmış bir sanal makine bytecode formatını (.opc) okuyarak çeşitli istatistiksel ve yapısal analizler gerçekleştiren uygulama, OOP konseptlerinin gerçek bir projede nasıl bir araya getirilebileceğini somut olarak göstermektedir.

Çalışmanın temel katkıları şöyle özetlenebilir: (1) 25 opcode içeren ve beş kategoriye ayrılan özel bir talimat seti mimarisi, (2) Factory, Singleton ve Strategy tasarım örüntülerini kullanan kapsamlı bir sınıf hiyerarşisi, (3) Shannon entropi tabanlı istatistiksel analiz, (4) iki dosya karşılaştırma ve raporlama altyapısı, (5) açık/koyu tema destekli modern Qt 6 arayüzü.

II. SİSTEM TASARIMI

A. Bytecode Formatı (.opc)

Araç, aşağıdaki ikili dosya formatını işlemektedir:

```
[4 byte magic: 0x4F504331 "OPC1"]
[2 byte versiyon: uint16_t]
[4 byte talimat sayısı: uint32_t]
[N x 4 byte: opcode | reg1 | reg2 | imm]
```

Her talimat dört byte'tan oluşmaktadır: opcode (1), kaynak kaydedici 1 (1), kaynak kaydedici 2 (1) ve anlık değer (1). Bu kompakt format, hem disassembly hem de istatistiksel analiz için yeterli bilgi içermektedir.

B. Instruction Sınıf Hiyerarşisi

Sistemin çekirdeği, soyut temel sınıf Instruction etrafında inşa edilmiştir. Bu tasarım, OOP'nin en temel ilkelerini —soyutlama, kalıtım ve polimorfizm— doğrudan yansıtmaktadır. Şekil 1'de sınıf diyagramı verilmektedir.

TABLO I. INSTRUCTION SINIF HİYERARŞİSİ

Sınıf	Kapsadığı Opcode'lar	N
ArithmeticInstruction	ADD, SUB, MUL, DIV, MOD, INC, DEC	7
MemoryInstruction	LOAD, STORE, MOV, PUSH, POP	5
ControlFlowInstruction	JMP, JEQ, JNE, JLT, JGT, CALL, RET, HALT	8
LogicalInstruction	AND, OR, XOR, NOT, CMP	5
IOInstruction	IN, OUT	2

Her alt sınıf, name(), category() ve description() saf sanal fonksiyonlarını override etmektedir. Bu polimorfik yapı sayesinde, farklı talimat türleri tek bir Instruction* işaretçisi üzerinden homojen bir şekilde işlenebilmektedir.

C. OpcodeTable Singleton'ı

OpcodeTable sınıfı, Singleton tasarım örüntüsünü uygulayarak opcode baytından mnemonic, kategori, operand formatı ve açıklama bilgilerine erişim sağlayan merkezi bir kayıt defteri işlevi görmektedir. Tek örnek, statik yerel değişken tekniğiyle thread-safe biçimde oluşturulmaktadır.

D. InstructionFactory

Factory örüntüsünü uygulayan InstructionFactory, ham dört byte'ı alarak OpcodeTable üzerinden kategoriyi belirler ve uygun alt sınıfı unique_ptr olarak döndürür. Bu yaklaşım, nesne üretim mantığını kullanım noktasından ayırarak tek sorumluluk ilkesini (SRP) pekiştirmektedir.

III. ANALİZ KATMANI

A. Analyzer Sınıf Hiyerarşisi

Strategy örüntüsünü kullanan Analyzer soyut sınıfı, analyze(const Program&) saf sanal metodunu tanımlar. Dört somut uygulama farklı analiz stratejilerini hayata geçirmektedir:

- **FrequencyAnalyzer:** Opcode frekans dağılımı ve yüzdesel oranlar.
- **CategoryAnalyzer:** Beş kategoriye göre talimat sınıflandırması.
- **EntropyAnalyzer:** Shannon entropi hesaplama ($H = -\sum p(x) \log_2 p(x)$).
- **ComparisonAnalyzer:** İki programın opcode profilini karşılaştırır.

B. Report Sınıfı ve Operatör Aşırı Yükleme

Report sınıfı, analiz sonuçlarını kapsüllemekte ve iki önemli operatör aşırı yüklemesi içermektedir. operator+ iki raporu birleştirirken, operator<< standart akış çıktısı sağlamaktadır. Sınıf ayrıca toCSV() ve toHTML() metodlarıyla çoklu dışa aktarma biçimlerini desteklemektedir.

IV. KULLANICI ARAYÜZÜ

A. Qt 6 Tabanlı Mimari

Arayüz katmanı Qt 6 Widgets ve Qt Charts modülleri ile geliştirilmiştir. MainWindow, dört sekmeli QTabWidget üzerinden Özet, Disassembly, Grafikler ve Karşılaştırma görünümelerini sunar. Disassembly sekmesi, adres/hex/mnemonic/operand/kategori/açıklama sütunlarını içeren özel bir QAbstractTableModel ile gerçekleştirilmiştir.

B. Tema Yönetimi

ThemeManager sınıfı, Qt Resource System ile gömülü iki QSS dosyası (light.qss ve dark.qss) arasında anlık geçiş sağlamaktadır. Kullanıcının tercih ettiği tema QSettings mekanizmasıyla kalıcı olarak saklanmaktadır. Qt Charts için QChart::ChartThemeDark / ChartThemeLight grafik temalarına otomatik uyum sağlanmaktadır.

C. Grafik Analiz

ChartView bileşeni, Qt Charts kullanarak iki görselleştirme sunar: opcode frekans bar grafiği (en sık 15 opcode) ve kategori dağılımı pasta/halka grafiği. Her iki grafik de tema değişiminde dinamik olarak güncellenmektedir.

V. SONUÇLAR

Bu çalışmada sunulan Opcode Analiz Aracı, Nesneye Yönelik Programlama ilkelerinin kapsamlı biçimde uygulandığı, çalışan bir C++ projesi ortaya koymaktadır. Soyut sınıflar (Instruction, Analyzer), kalıtım hiyerarşileri, polimorfik dispatch, RAII ile bellek yönetimi, Factory/Singleton/Strategy tasarım örüntüleri ve operatör aşırı yükleme — tüm bu OOP kavramları işlevsel bir yazılım çerçevesinde bir araya getirilmiştir.

Shannon entropi analizi, basit bir bytecode okuyucusunun ötesine geçerek kod yoğunluğu ve çeşitliliği hakkında anlamlı metrikler üretmektedir. İki dosya karşılaştırma özelliği, farklı program profillerinin sistematik değerlendirilmesine olanak tanımaktadır. Qt 6 ile gerçekleştirilen modern arayüz, açık/koyu tema desteği ve çoklu dışa aktarma seçenekleriyle kullanılabilirlik açısından da olgunlaşmış bir araç sunmaktadır.

Gelecek çalışmalar kapsamında gerçek x86 binary desteği, eklenti tabanlı Analyzer mimarisi ve karşılaştırmalı görselleştirme grafikleri planlanmaktadır.

KAYNAKLAR

- [1] B. Stroustrup, *The C++ Programming Language*, 4th ed. Addison-Wesley, 2013.
- [2] E. Gamma, R. Helm, R. Johnson, J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [3] Qt Group, *Qt 6 Documentation*, 2024. [Online]. Available: <https://doc.qt.io/qt-6/>
- [4] C. E. Shannon, "A mathematical theory of communication," *Bell System Technical Journal*, vol. 27, pp. 379–423, 1948.
- [5] *ISO/IEC 14882:2017, Programming Language — C++*, International Organization for Standardization, Geneva, 2017.

Opcode Analyzer: Virtual Machine Bytecode Analysis Using Object-Oriented Programming Principles

Mert Yasin YILMAZ

Department of Computer Engineering
Kastamonu University, Kastamonu, Turkey

Abstract

This paper presents an object-oriented opcode analysis tool developed using C++17 and the Qt 6 framework. The tool analyzes a custom virtual machine bytecode format (.opc) and demonstrates core OOP principles — encapsulation, inheritance, polymorphism, and abstraction — through a layered architecture. The system defines a custom instruction set with 25 opcodes categorized into five groups; implements an Instruction class hierarchy employing Factory and Singleton design patterns; provides multiple analysis strategies (frequency, category, Shannon entropy, comparison) via the Strategy pattern; and exposes a Report class with operator overloading and CSV/HTML export. A modern Qt 6 graphical interface with light/dark theme support, interactive charts, and recent-files management completes the application.

Index Terms—Object-Oriented Programming, C++17, Qt 6, Opcode Analysis, Design Patterns, Bytecode, Shannon Entropy

I. INTRODUCTION

Object-oriented programming (OOP) is one of the foundational paradigms of modern software engineering. Multi-paradigm languages such as C++ offer a powerful platform that combines OOP principles — encapsulation, inheritance, polymorphism, and abstraction — with performance-oriented system programming.

This paper presents the "Opcode Analyzer" developed as a final project for an Object-Oriented Programming course. The application is built using C++17, the Qt 6 GUI framework, and the CMake build system. It reads a custom virtual machine bytecode format (.opc) and performs various statistical and structural analyses, concretely demonstrating how OOP concepts can be integrated in a real project.

The main contributions of this work are: (1) a custom instruction set architecture with 25 opcodes organized into five categories; (2) a comprehensive class hierarchy employing Factory, Singleton, and Strategy design patterns; (3) statistical analysis based on Shannon entropy; (4) dual-file comparison and reporting infrastructure; and (5) a modern Qt 6 interface with light/dark theme support.

II. SYSTEM DESIGN

A. Bytecode Format (.opc)

The tool processes the following binary file format: a 4-byte magic number (0x4F504331, "OPC1"), a 2-byte version field, a 4-byte instruction count, and $N \times 4$ -byte instruction records. Each instruction record encodes four fields: opcode (1 byte), source register 1 (1 byte), source register 2 (1 byte), and an immediate value (1 byte). This compact format contains sufficient information for both disassembly and statistical analysis.

B. Instruction Class Hierarchy

The core of the system is built around the abstract base class `Instruction`, which directly reflects the fundamental OOP principles of abstraction, inheritance, and polymorphism. Each subclass overrides the pure virtual functions `name()`, `category()`, and `description()`. Table I summarizes the five concrete instruction classes.

TABLE I. INSTRUCTION CLASS HIERARCHY

Class	Opcodes	N
ArithmeticInstruction	ADD, SUB, MUL, DIV, MOD, INC, DEC	7
MemoryInstruction	LOAD, STORE, MOV, PUSH, POP	5
ControlFlowInstruction	JMP, JEQ, JNE, JLT, JGT, CALL, RET, HALT	8
LogicalInstruction	AND, OR, XOR, NOT, CMP	5
IOInstruction	IN, OUT	2

This polymorphic design allows heterogeneous instruction objects to be processed uniformly through an `Instruction*` pointer, enabling the rest of the system to remain agnostic of concrete types.

C. OpcodeTable Singleton

The OpcodeTable class implements the Singleton design pattern, functioning as a central registry that maps opcode bytes to mnemonic strings, categories, operand formats, and descriptions. The single instance is created using the thread-safe Meyers Singleton idiom (static local variable).

D. InstructionFactory

InstructionFactory implements the Factory design pattern. It receives four raw bytes, queries OpcodeTable to determine the category, and returns the appropriate concrete subclass wrapped in a `std::unique_ptr`. This separates object construction from object use, reinforcing the Single Responsibility Principle (SRP).

III. ANALYSIS LAYER

A. Analyzer Class Hierarchy

The abstract Analyzer class defines the pure virtual method `analyze(const Program&)`, implementing the Strategy design pattern. Four concrete analyzers provide distinct strategies:

- **FrequencyAnalyzer:** Opcode frequency distribution and percentage ratios.
- **CategoryAnalyzer:** Instruction classification across five categories.
- **EntropyAnalyzer:** Shannon entropy calculation ($H = -\sum p(x) \log_2 p(x)$).
- **ComparisonAnalyzer:** Compares opcode profiles of two programs, computing per-opcode deltas and change percentages.

B. Report Class and Operator Overloading

The Report class encapsulates analysis results and demonstrates two operator overloads: `operator+` merges two reports (combining opcode frequencies and recalculating percentages), while `operator<<` provides standard stream output. The class further supports CSV and HTML export through `toCSV()` and `toHTML()` methods, enabling integration into external tools or browser-based presentation.

IV. USER INTERFACE

A. Qt 6 Architecture

The interface layer is implemented with Qt 6 Widgets and the Qt Charts module. MainWindow hosts a four-tab QTabWidget providing Summary, Disassembly, Charts, and Compare views. The Disassembly tab uses a custom QAbstractTableModel (DisassemblyModel) presenting address, hex dump, mnemonic, operands, category, and description columns with category-specific text coloring.

B. Theme Management

ThemeManager applies Qt Style Sheets (QSS) embedded via the Qt Resource System, enabling instant light-to-dark transitions with no flicker. The user's preference is persisted through QSettings. Qt Charts themes (ChartThemeDark / ChartThemeLight) are updated on each theme toggle to maintain visual consistency.

C. Chart Visualization

ChartView renders two interactive Qt Charts: a bar chart displaying the top-15 opcodes by frequency, and a donut pie chart showing category distribution. Both charts animate on data load and refresh dynamically on theme change.

V. CONCLUSION

The Opcode Analyzer presented in this work demonstrates a comprehensive application of Object-Oriented Programming principles in a fully functional C++ project. Abstract classes (Instruction, Analyzer), inheritance hierarchies, polymorphic dispatch, RAII-based memory management, three design patterns (Factory, Singleton, Strategy), and operator overloading are all integrated into a cohesive software architecture.

Shannon entropy analysis elevates the tool beyond a simple bytecode reader, generating meaningful metrics about code diversity and structure. The dual-file comparison feature enables systematic evaluation of different program profiles. The modern Qt 6 interface, with light/dark theme support and multiple export options, represents a deployable, user-facing application.

Future work includes support for real x86 binary formats via the Capstone disassembly library, a plugin-based Analyzer architecture for extensibility, and comparative visualization charts for multi-file analysis sessions.

REFERENCES

- [1] B. Stroustrup, *The C++ Programming Language*, 4th ed. Addison-Wesley, 2013.
- [2] E. Gamma, R. Helm, R. Johnson, J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [3] Qt Group, *Qt 6 Documentation*, 2024. [Online]. Available: <https://doc.qt.io/qt-6/>
- [4] C. E. Shannon, "A mathematical theory of communication," *Bell System Technical Journal*, vol. 27, pp. 379–423, 1948.
- [5] *ISO/IEC 14882:2017*, Programming Languages — C++. International Organization for Standardization, Geneva, 2017.